

Тема 3.4 Свойства и декораторы классов

Свойства класса

Внутри класса можно создать идентификатор, через который в дальнейшем будут производиться операции получения и изменения значения какого-либо атрибута, а также его удаления. Создается такой идентификатор с помощью функции **property()**. Формат функции:

```
<Свойство> = property(<Чтение>[, <Запись>[, <Удаление>[, <Строка документирования>]])
```

В первых трех параметрах указывается ссылка на соответствующий метод класса. При попытке получить значение будет вызван метод, указанный в первом параметре. При операции присваивания значения будет вызван метод, указанный во втором параметре, - этот метод должен принимать один параметр. В случае удаления атрибута вызывается метод, указанный в третьем параметре. Если в качестве какого-либо параметра задано значение None, то это означает, что соответствующий метод не поддерживается. Рассмотрим свойства класса на примере:

```
class MyClass:
    def __init__(self, value):
        self.__var = value
    def get_var(self):      # Чтение
        return self.__var
    def set_var(self, value): # Запись
        self.__var = value
    def del_var(self):     # Удаление
        del self.__var
    v = property(get_var, set_var, del_var, "Строка документирования")
c = MyClass(5)
c.v = 35 # Вызывается метод set_var()
print(c.v)      # Вызывается метод get_var()
del c.v         # Вызывается метод del_var()
```

Результат работы приложения:

35

Python поддерживает альтернативный метод определения свойств - с помощью методов **property()**, **setter()** и **deleter()**, которые используются в декораторах. Пример их использования приведен ниже:

```
class MyClass:
    def __init__(self, value):
        self.__var = value
    @property
    def v(self):      # Чтение
        return self.__var
    @v.setter
    def v(self, value): # Запись
```

```

        self.__var = value
    @v.deleter
    def v(self):      # Удаление
        del self.__var
c = MyClass(5)
c.v = 35 # Чтение
print(c.v)      # Запись
del c.v        # Удаление

```

Результат работы приложения:

35

Имеется возможность определить абстрактное свойство - в этом случае все реализующие его методы должны быть переопределены в подклассе. Выполняется это с помощью декоратора **@abstractmethod** из модуля **abc**. Пример определения абстрактного свойства показан ниже:

```

from abc import ABCMeta, abstractmethod
class MyClass1 (metaclass=ABCMeta):
    def __init__(self, value):
        self.__var = value
    @property
    @abstractmethod
    def v(self):      # Чтение
        return self.__var
    @v.setter
    @abstractmethod
    def v(self, value): # Запись
        self.__var = value
    @v.deleter
    @abstractmethod
    def v(self):      # Удаление
        del self.__var

```

Создание свойств

Получить доступ к атрибуту, в том числе закрытому, в Python можно с помощью свойств. Чтобы создать свойство, следует написать метод, который вернет значение, однако, перед тем как метод будет объявлен, надо написать декоратор **@property**. Продемонстрируем создание свойства на примере, для чего напишем метод **vozs**, с помощью которого получим доступ к закрытому атрибуту **_vozs**,

Метод **vozs** ниже предворяем декоратором **@property**, а имя свойства будет совпадать с именем метода.

```

class Student:
    def __init__(self, name, city, vozs):
        self.name = name
        self.city = city
        self._vozs = vozs # используйте приватное поле с одним подчеркиванием

```

```

@property
def vozr(self):
    return self._vozr # возвращайте приватное поле
def message(self):
    print("Здравствуйте, я студент", self.name, "из", self.city, "возраст", self.vozr)
stl = Student("Сергей", "Москва", "17")
stl.message()

```

Теперь через свойство **vozr** получить доступ к значению атрибута внутри или вне объявления класса. Непосредственно в коде оператором **self.vozr** вызывается метод **vozr**, который вернет атрибут **__vozr**. Происходит обращение к методу **vozr**, которое возвратит значение **__vozr**, т. е. число 17.

Декораторы классов в Python

В языке **Python**, помимо декораторов функций, поддерживаются также декораторы классов, которые позволяют изменить поведение самих классов. В качестве параметра декоратор принимает ссылку на объект класса, поведение которого необходимо изменить, и должен возвращать ссылку на тот же класс или какой-либо другой. Пример декорирования класса показан ниже:

```

def deco(C):                # Принимает объект класса
    print("Внутри декоратора") # Производит какие-то действия
    return C                 # Возвращает объект класса

```

```

@deco
class MyClass:
    def __init__(self, value):
        self.v = value

```

```

c = MyClass(5)

```

```

print(c.v)

```

Результат работы приложения:

Внутри декоратора

5

Для того, чтобы понять, как работают декораторы, в первую очередь следует вспомнить, что функции в **python** являются объектами, соответственно, их можно возвращать из другой функции или передавать в качестве аргумента.

Декораторы — это, по сути, "обёртки", которые дают возможность изменить поведение функции, не изменяя её код.

Создадим свой декоратор «вручную»:

```

def my_shiny_new_decorator(function_to_decorate):

```

Внутри себя декоратор определяет функцию-"обёртку". Она будет обёрнута вокруг декорируемой,

получая возможность исполнять произвольный код до и после неё.

```

def the_wrapper_around_the_original_function():

```

```

    print("Я - код, который отработает до вызова функции")

```

```

    function_to_decorate() # Сама функция

```

```
    print("А я - код, срабатывающий после")
# Вернём эту функцию
    return the_wrapper_around_the_original_function
# Представим теперь, что есть функция, которую не планируется больше трогать.
```

```
def stand_alone_function():
    print("Я простая одинокая функция, ты ведь не посмеешь меня изменять?")
stand_alone_function()
Я простая одинокая функция, ты ведь не посмеешь меня изменять?
# Однако, чтобы изменить её поведение, нужно декорировать её, то есть просто передать декоратору,
# который обернет исходную функцию в любой код, который потребуется, и вернёт новую,
# готовую к использованию функцию:
stand_alone_function_decorated = my_shiny_new_decorator(stand_alone_function)
stand_alone_function_decorated()
```

Результат выполнения:

```
Я - код, который отработает до вызова функции
Я простая одинокая функция, ты ведь не посмеешь меня изменять?
А я - код, срабатывающий после
Наверное, теперь хотелось бы, чтобы каждый раз, во время вызова stand_alone_function, вместо неё вызывалась stand_alone_function_decorated. Для этого просто перезапишем stand_alone_function:
stand_alone_function = my_shiny_new_decorator(stand_alone_function)
stand_alone_function()
Я - код, который отработает до вызова функции
Я простая одинокая функция, ты ведь не посмеешь меня изменять?
А я - код, срабатывающий после
Собственно, это и есть декораторы. Вот так можно было записать предыдущий пример, используя синтаксис декораторов:
```

```
@my_shiny_new_decorator
def another_stand_alone_function():
    print("Оставь меня в покое")
another_stand_alone_function()
```

Результат выполнения:

```
Я - код, который отработает до вызова функции
Оставь меня в покое
А я - код, срабатывающий после
То есть, декораторы в python — это просто синтаксический сахар для конструкций вида:
```

```
another_stand_alone_function
my_shiny_new_decorator(another_stand_alone_function) =
```

При этом, естественно, можно использовать несколько декораторов для одной функции, например так:

```
def bread(func):
```

```

def wrapper():
    print()
    func()
    print("<\_____/>")
    return wrapper
def ingredients(func):
    def wrapper():
        print("#помидоры#")
        func()
        print("~салат~")
    return wrapper
def sandwich(food="--ветчина--"):
    print(food)
sandwich()
--ветчина--
sandwich = bread(ingredients(sandwich))
sandwich()
#помидоры#
--ветчина--
~салат~
<\_____/>

```

И используя синтаксис декораторов:

```

@bread
@ingredients
def sandwich(food="--ветчина--"):
    print(food)
sandwich()
#помидоры#
--ветчина--
~салат~
<\_____/>

```

Также нужно помнить о том, что важен порядок декорирования. Сравните с предыдущим примером:

```

@ingredients
@bread
def sandwich(food="--ветчина--"):
    print(food)
sandwich()
#помидоры#
--ветчина--
<\_____/>
~салат~

```

Определение декоратора как класса в Python

Декораторы - очень мощный и полезный инструмент в Python, поскольку он позволяет программистам изменять поведение функции или класса. Декораторы позволяют обернуть другую функцию, чтобы расширить поведение обернутой функции, не изменяя ее постоянно.

Можно определить декоратор как класс, чтобы сделать это, необходимо использовать метод классов `__call__`. Когда пользователю необходимо создать объект, который действует как функция, тогда декоратору функции необходимо вернуть объект, который действует как функция, поэтому `__call__` может быть полезен. **Например:**

```
class MyDecorator:
    def __init__(self, function):
        self.function = function
    def __call__(self):
        self.function()
@MyDecorator
def function():
    print("GeeksforGeeks")
function()
```

Результат:

GeeksforGeeks

Класс Decorator с *args и **kwargs в Python

Чтобы использовать декоратор класса с аргументом *args и **kwargs нужно использовать функцию `__call__` и передать оба аргумента в заданную функцию:

```
class MyDecorator:
    def __init__(self, function):
        self.function = function
    def __call__(self, *args, **kwargs):
        self.function(*args, **kwargs)
@MyDecorator
def function(name, message='Hello'):
    print("{} {}".format(message, name))
function("geeks_for_geeks", "hello")
```

Результат:

привет, geeks_for_geeks

Класс Decorator с оператором return в Python

В данном примере функции ничего не возвращали, поэтому проблем не возникает, но может потребоваться возвращаемое значение. Поэтому используем оператор `return` с декоратором класса. **Например:**

```
class SquareDecorator:
    def __init__(self, function):
        self.function = function
    def __call__(self, *args, **kwargs):
```

```

        result = self.function(*args, **kwargs)
        return result
@SquareDecorator
def get_square(n):
    print("given number is:", n)
    return n * n
print("Square of number is:", get_square(195))
Результат: данный номер: 195
Площадь номера: 38025

```

Передача декоратором аргументов в функцию

Однако, все декораторы, которые рассматривались, не имели одного очень важного функционала — передачи аргументов декорируемой функции. Собственно, это тоже несложно сделать.

```

def a_decorator_passing_arguments(function_to_decorate):
    def a_wrapper_accepting_arguments(arg1, arg2):
        print("Смотри, что я получил:", arg1, arg2)
        function_to_decorate(arg1, arg2)
    return a_wrapper_accepting_arguments
# Теперь, когда вызовем функцию, которую возвращает декоратор, вызываем её
"обёртку",
# передаём ей аргументы и уже в свою очередь она передаёт их декорируемой
функции
@a_decorator_passing_arguments
def print_full_name(first_name, last_name):
    print("Меня зовут", first_name, last_name)
print_full_name("Vasya", "Pupkin")
Смотри, что я получил: Vasya Pupkin
Меня зовут Vasya Pupkin

```

Декорирование методов

Один из важных фактов, которые следует понимать, заключается в том, что функции и методы в Python — это практически одно и то же, за исключением того, что методы всегда ожидают первым параметром ссылку на сам объект (self). Это значит, что можно создавать декораторы для методов точно так же, как и для функций, просто не забывая про self.

```

def method_friendly_decorator(method_to_decorate):
    def wrapper(self, lie):
        lie -= 3
        return method_to_decorate(self, lie)
    return wrapper
class Lucy:
    def __init__(self):

```

```

    self.age = 32
    @method_friendly_decorator
    def sayYourAge(self, lie):
        print("Мне {} лет, а ты бы сколько дал?".format(self.age + lie))
l = Lucy()
l.sayYourAge(-3)

```

Мне 26 лет, а ты бы сколько дал?

Конечно, если создать максимально общий декоратор и хотим, чтобы его можно было применить к любой функции или методу, то можно воспользоваться распаковкой аргументов:

```

def a_decorator_passing_arbitrary_arguments(function_to_decorate):
    # Данная "обёртка" принимает любые аргументы
    def a_wrapper_accepting_arbitrary_arguments(*args, **kwargs):
        print("Передали ли мне что-нибудь?:")
        print(args)
        print(kwargs)
        function_to_decorate(*args, **kwargs)
    return a_wrapper_accepting_arbitrary_arguments

```

```

@a_decorator_passing_arbitrary_arguments

```

```

def function_with_no_argument():
    print("Python is cool, no argument here.")

```

```

function_with_no_argument()

```

Передали ли мне что-нибудь?:

```
()
```

```
{}
```

Python is cool, no argument here.

```

@a_decorator_passing_arbitrary_arguments

```

```

def function_with_arguments(a, b, c):

```

```
    print(a, b, c)

```

```

function_with_arguments(1, 2, 3)

```

Передали ли мне что-нибудь?:

```
(1, 2, 3)
```

```
{}
```

```
1 2 3
```

```

@a_decorator_passing_arbitrary_arguments

```

```

def function_with_named_arguments(a, b, c, platypus="Почему нет?"):

```

```
    print("Любят ли {}, {} и {} утконосов? {}".format(a, b, c, platypus))

```

```

function_with_named_arguments("Билл", "Линус", "Стив",

```

```
platypus="Определенно!")

```

Передали ли мне что-нибудь?:

```
('Билл', 'Линус', 'Стив')
```

```
{'platypus': 'Определенно!'}
```

Любят ли Билл, Линус и Стив утконосов? Определенно!

```

class Mary(object):

```

```
    def __init__(self):

```

```

    self.age = 31
    @a_decorator_passing_arbitrary_arguments
    def sayYourAge(self, lie=-3): # Теперь можно указать значение по умолчанию
        print("Мне {} лет, а ты бы сколько дал?".format(self.age + lie))
m = Mary()
m.sayYourAge()
Передали ли мне что-нибудь?:
(<__main__.Mary object at 0x7f6373017780>,)
{}
Мне 28 лет, а ты бы сколько дал?

```

Декораторы с аргументами

А теперь попробуем написать декоратор, принимающий аргументы:

```

def decorator_maker():
    print("Я создаю декораторы! Я буду вызван только раз: когда ты попросишь
меня создать декоратор.")
    def my_decorator(func):
        print("Я - декоратор! Я буду вызван только раз: в момент декорирования
функции.")
        def wrapped():
            print ("Я - обёртка вокруг декорируемой функции.\n"
                  "Я буду вызвана каждый раз, когда ты вызываешь декорируемую
функцию.\n"
                  "Я возвращаю результат работы декорируемой функции.")
            return func()
        print("Я возвращаю обёрнутую функцию.")
        return wrapped
    print("Я возвращаю декоратор.")
    return my_decorator

# Давайте теперь создадим декоратор. Это всего лишь ещё один вызов функции
new_decorator = decorator_maker()
Я создаю декораторы! Я буду вызван только раз: когда ты попросишь меня
создать декоратор.
Я возвращаю декоратор.
# Теперь декорируем функцию
def decorated_function():
    print("Я - декорируемая функция.")
decorated_function = new_decorator(decorated_function)
Я - декоратор! Я буду вызван только раз: в момент декорирования функции.
Я возвращаю обёрнутую функцию.
# Теперь наконец вызовем функцию:
decorated_function()
Я - обёртка вокруг декорируемой функции.
Я буду вызвана каждый раз, когда ты вызываешь декорируемую функцию.

```

Я возвращаю результат работы декорируемой функции.

Я - декорируемая функция.

Теперь перепишем данный код с помощью декораторов:

```
@decorator_maker()
```

```
def decorated_function():
```

```
    print("Я - декорируемая функция.")
```

Я создаю декораторы! Я буду вызван только раз: когда ты попросишь меня создать декоратор.

Я возвращаю декоратор.

Я - декоратор! Я буду вызван только раз: в момент декорирования функции.

Я возвращаю обёрнутую функцию.

```
decorated_function()
```

Я - обёртка вокруг декорируемой функции.

Я буду вызвана каждый раз когда ты вызываешь декорируемую функцию.

Я возвращаю результат работы декорируемой функции.

Я - декорируемая функция.

Вернёмся к аргументам декораторов, ведь, если использовать функцию, чтобы создавать декораторы "на лету", можно передавать ей любые аргументы, верно?

```
def decorator_maker_with_arguments(decorator_arg1, decorator_arg2):
```

```
    print("Я создаю декораторы! И я получил следующие аргументы:",  
          decorator_arg1, decorator_arg2)
```

```
    def my_decorator(func):
```

```
        print("Я - декоратор. И ты всё же смог передать мне эти аргументы:",  
              decorator_arg1, decorator_arg2)
```

```
        # Не перепутайте аргументы декораторов с аргументами функций!
```

```
        def wrapped(function_arg1, function_arg2):
```

```
            print ("Я - обёртка вокруг декорируемой функции.\n"
```

```
                  "И я имею доступ ко всем аргументам\n"
```

```
                  "\t- и декоратора: {0} {1}\n"
```

```
                  "\t- и функции: {2} {3}\n"
```

```
                  "Теперь я могу передать нужные аргументы дальше"
```

```
                  .format(decorator_arg1, decorator_arg2,  
                          function_arg1, function_arg2))
```

```
            return func(function_arg1, function_arg2)
```

```
        return wrapped
```

```
    return my_decorator
```

```
@decorator_maker_with_arguments("Леонард", "Шелдон")
```

```
def decorated_function_with_arguments(function_arg1, function_arg2):
```

```
    print ("Я - декорируемая функция и я знаю только о своих аргументах: {0}"  
          " {1}".format(function_arg1, function_arg2))
```

Я создаю декораторы! И я получил следующие аргументы: Леонард Шелдон

Я - декоратор. И ты всё же смог передать мне эти аргументы: Леонард Шелдон

```
decorated_function_with_arguments("Раджеш", "Говард")
```

Я - обёртка вокруг декорируемой функции.

И я имею доступ ко всем аргументам

- и декоратора: Леонард Шелдон
- и функции: Раджеш Говард

Теперь я могу передать нужные аргументы дальше

Я - декорируемая функция и я знаю только о своих аргументах: Раджеш Говард

Таким образом, можно передавать декоратору любые аргументы, как обычной функции. Необходимо использовать и распаковку через `*args` и `**kwargs` в случае необходимости.

Некоторые особенности работы с декораторами

Декораторы несколько замедляют вызов функции, не забывайте об этом.

Нельзя «раздекорировать» функцию. Безусловно, существуют трюки, позволяющие создать декоратор, который можно отсоединить от функции, но это плохая практика. Правильнее будет запомнить, что, если функция декорирована — это не отменить.

Декораторы оборачивают функции, что может затруднить отладку.

Последняя проблема частично решена добавлением в модуле `functools` функции `functools.wraps`, копирующей всю информацию об оборачиваемой функции (её имя, из какого она модуля, её документацию и т.п.) в функцию-обёртку.

Забавным фактом является то, что `functools.wraps` тоже является декоратором.

```
def foo():
    print("foo")
print(foo.__name__)
foo
# Однако, декораторы мешают нормальному ходу дел:
def bar(func):
    def wrapper():
        print("bar")
        return func()
    return wrapper
@bar
def foo():
    print("foo")
print(foo.__name__)
wrapper
import functools # "functools" может с этим помочь
def bar(func):
    # Объявляем "wrapper" оборачивающим "func"
    # и запускаем магию:
    @functools.wraps(func)
    def wrapper():
        print("bar")
        return func()
    return wrapper
@bar
```

```
def foo():
    print("foo")
print(foo.__name__)
foo
```

Примеры использования декораторов

Декораторы могут быть использованы для расширения возможностей функций из сторонних библиотек (код которых нельзя изменять), или для упрощения отладки (не хотим изменять код, который ещё не устоялся).

Также полезно использовать декораторы для расширения различных функций одним и тем же кодом, без повторного его переписывания каждый раз, например:

```
def benchmark(func):
    """
    Декоратор, выводящий время, которое заняло
    выполнение декорируемой функции.
    """
    import time
    def wrapper(*args, **kwargs):
        t = time.clock()
        res = func(*args, **kwargs)
        print(func.__name__, time.clock() - t)
        return res
    return wrapper

def logging(func):
    """
    Декоратор, логирующий работу кода.
    (хорошо, он просто выводит вызовы, но тут могло быть и логирование!)
    """
    def wrapper(*args, **kwargs):
        res = func(*args, **kwargs)
        print(func.__name__, args, kwargs)
        return res
    return wrapper

def counter(func):
    """
    Декоратор, считающий и выводящий количество вызовов
    декорируемой функции.
    """
    def wrapper(*args, **kwargs):
        wrapper.count += 1
        res = func(*args, **kwargs)
        print("{0} была вызвана: {1}x".format(func.__name__, wrapper.count))
        return res
    wrapper.count = 0
```

```

    return wrapper
@benchmark
@logging
@counter
def reverse_string(string):
    return ".join(reversed(string))
print(reverse_string("A роза упала на лапу Азора"))
reverse_string была вызвана: 1x
wrapper ('A роза упала на лапу Азора',) {}
wrapper 0.000117999999999997923
арозА упал ан алапу азор А
print(reverse_string("A man, a plan, a canoe, pasta, heros, rajahs, a coloratura,"
"a maps, snipe, percale, macaroni, a gag, a banana bag, a tan, a tag,"
"a banana bag again (or a camel), a crepe, pins, Spam, a rut, a Rolo, cash,"
"a jar, sore hats, a peon, a canal: Panama!"))
reverse_string была вызвана: 2x
wrapper ('A man, a plan, a canoe, pasta, heros, rajahs, a coloratura,maps, snipe, ...',)
{}

```